

Prueba de Caja Blanca

“Sistema de Gestión de Caja para Tags y Productos”

Integrantes:
Leopoldo Bolaños
Matias Medina
Martin Palacios

Fecha: 2026/06/13

REQ008 (Método de Pago):

Implementar una sección dentro del sistema que permita visualizar el resumen de la cuenta actual, seleccionar la modalidad de cobro (Efectivo, Tarjeta, Transferencia o Mixto) y confirmar la transacción para enviar los montos calculados al flujo de caja diaria.

1. CÓDIGO FUENTE para Método de Pago

Controlador (Código)

```
package espe.edu.ec.programacaja.controller;

import espe.edu.ec.programacaja.model.CashFlowModel;
import espe.edu.ec.programacaja.model.CurrentSaleModel;
import espe.edu.ec.programacaja.model.PaymentDetail;
import espe.edu.ec.programacaja.model.PaymentMethod;
import espe.edu.ec.programacaja.view.PaymentPanel;
import java.math.BigDecimal;
import java.util.Locale;
import java.util.regex.Pattern;
import javax.swing.JOptionPane;

public class PaymentController {

    private static final Pattern MONEY_PATTERN = Pattern.compile("^\\d+(\\.\\d{1,2})?$");

    private final PaymentPanel view;
    private final CurrentSaleModel currentSaleModel;
    private final CashFlowModel cashFlowModel;

    public PaymentController(PaymentPanel view, CurrentSaleModel currentSaleModel, CashFlowModel cashFlowModel) {
        this.view = view;
        this.currentSaleModel = currentSaleModel;
        this.cashFlowModel = cashFlowModel;

        initController();
        refreshSaleSummary();
    }

    private void initController() {
        view.getBtnRefreshSale().addActionListener(e -> refreshSaleSummary());
        view.getBtnConfirmPayment().addActionListener(e -> confirmPayment());
        view.getBtnClearSale().addActionListener(e -> clearCurrentSale());
    }

    public void refreshSaleSummary() {
        view.setSaleSummary(currentSaleModel.getReceiptText());
    }

    private void confirmPayment() {
        refreshSaleSummary();

        if (!currentSaleModel.hasItems()) {
            JOptionPane.showMessageDialog(
                view,
                "No existen productos o servicios en la venta actual.\n"
                + "Si ya seleccionó productos o servicios, primero presione "
                + "\"Agregar a venta actual\" en el módulo correspondiente.\n"
            );
        }
    }
}
```

```

        + "Luego presione \"Actualizar venta\" o vuelva a ingresar a esta pestaña.",
        "Venta vacía",
        JOptionPane.WARNING_MESSAGE
    );
    return;
}

try {
    PaymentDetail paymentDetail = buildPaymentDetail();

    String message = currentSaleModel.getReceiptText()
        + "\n"
        + paymentDetail.getPaymentSummaryText()
        + "\n¿Desea confirmar el pago y enviar esta venta a caja?";

    int option = JOptionPane.showConfirmDialog(
        view,
        message,
        "Confirmar pago",
        JOptionPane.YES_NO_OPTION,
        JOptionPane.QUESTION_MESSAGE
    );

    if (option != JOptionPane.YES_OPTION) {
        return;
    }

    cashFlowModel.addPaidSale(currentSaleModel, paymentDetail);

    currentSaleModel.clearSale();
    view.clearPaymentFields();
    refreshSaleSummary();

    JOptionPane.showMessageDialog(
        view,
        "Pago confirmado correctamente.\n"
        + "La venta fue enviada al cierre de caja.",
        "Pago confirmado",
        JOptionPane.INFORMATION_MESSAGE
    );

} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(
        view,
        ex.getMessage(),
        "Error en el pago",
        JOptionPane.ERROR_MESSAGE
    );
}

private PaymentDetail buildPaymentDetail() {
    PaymentMethod selectedMethod = view.getSelectedPaymentMethod();
    double saleTotal = roundToTwoDecimals(currentSaleModel.getTotal());

    if (selectedMethod == null) {

```

```

        throw new IllegalArgumentException("Debe seleccionar un método de pago.");
    }

    if (selectedMethod == PaymentMethod.MIXED) {
        validateEmptyMixedFields();

        double cashAmount = parseMoneyAmount(view.getCashAmountText(), "efectivo");
        double transferAmount = parseMoneyAmount(view.getTransferAmountText(), "transferencia");
        double cardAmount = parseMoneyAmount(view.getCardAmountText(), "tarjeta");

        view.setFormattedMixedAmounts(cashAmount, transferAmount, cardAmount);

        return PaymentDetail.createMixedPayment(
            saleTotal,
            cashAmount,
            transferAmount,
            cardAmount
        );
    }

    return PaymentDetail.createSinglePayment(selectedMethod, saleTotal);
}

private void validateEmptyMixedFields() {
    boolean cashEmpty = view.getCashAmountText().isEmpty();
    boolean transferEmpty = view.getTransferAmountText().isEmpty();
    boolean cardEmpty = view.getCardAmountText().isEmpty();

    if (cashEmpty && transferEmpty && cardEmpty) {
        throw new IllegalArgumentException("Debe ingresar los valores del pago mixto.");
    }

    if (cashEmpty) {
        throw new IllegalArgumentException("El campo efectivo es obligatorio.");
    }

    if (transferEmpty) {
        throw new IllegalArgumentException("El campo transferencia es obligatorio.");
    }

    if (cardEmpty) {
        throw new IllegalArgumentException("El campo tarjeta es obligatorio.");
    }
}

private double parseMoneyAmount(String text, String fieldName) {
    String normalizedText = text.trim().replace(",", ".");

    if (!MONEY_PATTERN.matcher(normalizedText).matches()) {
        throw new IllegalArgumentException(
            "El valor de " + fieldName + " debe ser numérico y tener máximo 2 decimales. Ejemplo:
7.20"
        );
    }

    BigDecimal amount = new BigDecimal(normalizedText);

```

```

        if (amount.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("El valor de " + fieldName + " no puede ser negativo.");
        }

        return amount.setScale(2).doubleValue();
    }

    private double roundToTwoDecimals(double value) {
        return new BigDecimal(String.format(Locale.US, "%.2f", value)).doubleValue();
    }

    private void clearCurrentSale() {
        if (!currentSaleModel.hasItems()) {
            JOptionPane.showMessageDialog(
                view,
                "No existen productos o servicios para limpiar.",
                "Venta vacía",
                JOptionPane.INFORMATION_MESSAGE
            );
            return;
        }

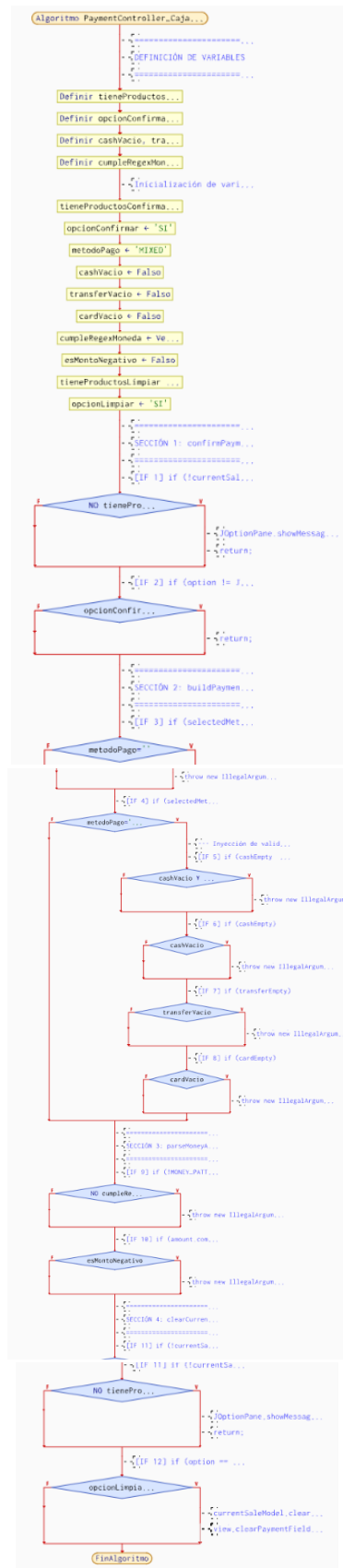
        int option = JOptionPane.showConfirmDialog(
            view,
            "¿Está seguro de limpiar la venta actual?",
            "Confirmar limpieza",
            JOptionPane.YES_NO_OPTION
        );

        if (option == JOptionPane.YES_OPTION) {
            currentSaleModel.clearSale();
            view.clearPaymentFields();
            refreshSaleSummary();

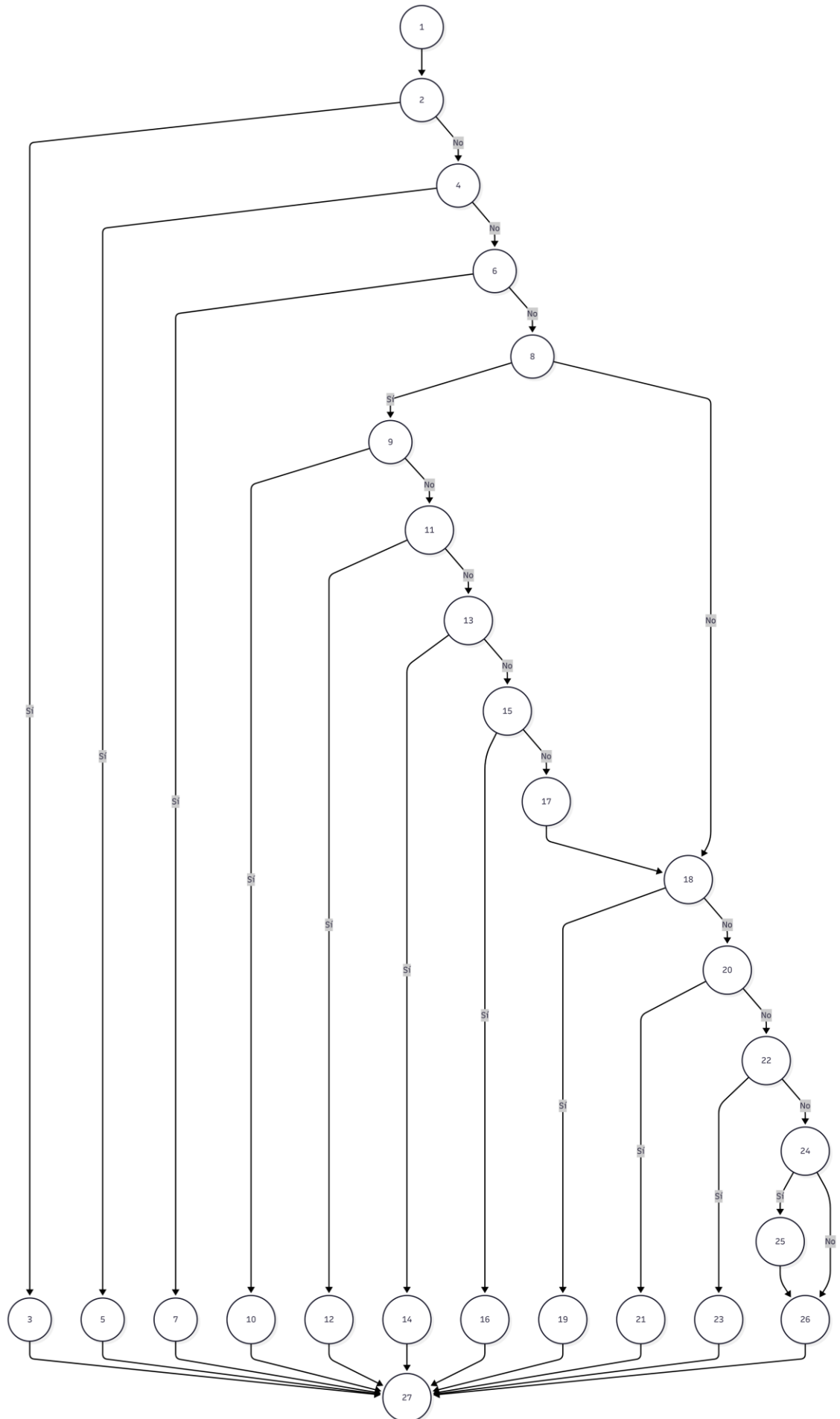
            JOptionPane.showMessageDialog(
                view,
                "La venta actual fue limpiada correctamente.",
                "Venta limpiada",
                JOptionPane.INFORMATION_MESSAGE
            );
        }
    }
}

```

2. DIAGRAMA DE FLUJO (DF) Método de Pago



3. GRAFO DE FLUJO (GF) Método de Pago



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) Método de Pago

Determinar en base al GF del numeral 4

RUTAS

R1: 1-2-4-6-8-18-20-22-24-26-27

R2: 1-2-3-27

R3: 1-2-4-5-27

R4: 1-2-4-6-7-27

R5: 1-2-4-6-8-9-10-27

R6: 1-2-4-6-8-9-11-12-27

R7: 1-2-4-6-8-9-11-13-14-27

R8: 1-2-4-6-8-9-11-13-15-16-27

R9: 1-2-4-6-8-9-11-13-15-17-18-19-27

R10: 1-2-4-6-8-9-11-13-15-17-18-20-21-27

R11: 1-2-4-6-8-18-20-22-23-27

R12: 1-2-4-6-8-18-20-22-24-25-26-27

R13: 1-2-4-6-8-9-11-13-15-17-18-20-22-24-26-27

5. COMPLEJIDAD CICLOMÁTICA Método de Pago

Se puede calcular de las siguientes formas:

V(G) Nodos predicados (IF-THEN-ELSE) +1

$V(G) = 12 + 1$

$V(G) = 13$

$V(G) = A - N + 2$

$V(G) = 38 - 27 + 2$

$V(G) = 4$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos